MISSION DOCKER

Mission Docker - Monitoring & Sécurité - Stage BTS SIO (1 mois)

Contexte de la mission :

L'entreprise souhaite mettre en place une infrastructure de monitoring et de sécurité conteneurisée. En tant que stagiaire, vous allez découvrir et déployer des outils professionnels utilisés en entreprise, le tout via Docker pour simplifier l'installation.

Objectifs pédagogiques :

- Maîtriser les bases de Docker et Docker Compose
- Déployer des solutions de monitoring professionnelles
- Comprendre l'utilité de chaque outil
- Documenter les déploiements réalisés

Prérequis techniques :

- PC avec 8GB RAM minimum (16GB recommandé)
- Windows 10/11 Pro ou Linux
- Docker Desktop installé
- Connexion internet stable (téléchargements importants)

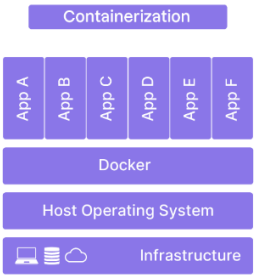
Planning sur 4 semaines :

Semaine 1 : Docker basics et Uptime Kuma (monitoring simple)

Jour 1-2 : Découverte de Docker

- **Concept de base :**

  - **Qu'est-ce qu'un conteneur ?**
    La conteneurisation :
    La conteneurisation est une technique permettant de créer des "conteneurs", groupes d'un ou plusieurs processus, qui **partagent le même noyau** du système d'exploitation mais s'exécutent de manière isolée les uns des autres.



Fonctionnement

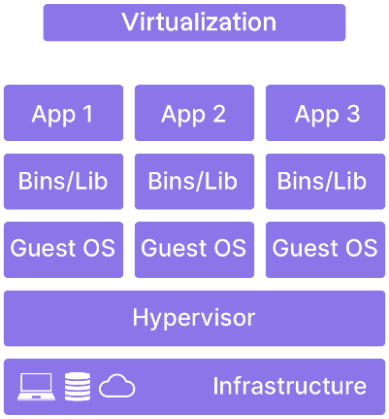
Les conteneurs sont gérés par des moteurs de conteneurisation comme Docker.
Ces moteurs utilisent des fonctionnalités du noyau Linux telles que les **cgroups** et les

namespaces pour isoler et contrôler les ressources. **Les conteneurs partagent les ressources du système hôte**, permettant généralement une gestion plus dynamique et efficace de celles-ci.

- **Différence avec une VM :**

La virtualisation

La virtualisation est une technologie qui permet de créer plusieurs machines virtuelles (VM) sur un seul serveur physique. **Chaque machine virtuelle fonctionne comme une machine indépendante** avec son propre système d'exploitation, ses applications et ses ressources dédiées.



Fonctionnement

La clé de la virtualisation est l'hyperviseur, un logiciel qui permet de créer et de gérer des machines virtuelles. Il existe deux types d'hyperviseurs :

- **Hyperviseur de type 1** ("Bare metal") : Ce type d'hyperviseur fonctionne directement sur le matériel. Exemples: VMware ESXi, Microsoft Hyper-V, Xen.
- **Hyperviseur de type 2** ("Hosted") : Ce type d'hyperviseur fonctionne sur un système d'exploitation hôte. Exemples: VMWare Workstation, Oracle VM Virtualbox.

- **Les commandes essentielles :**

```
docker run hello-world              # Test
docker ps                           # Voir les conteneurs
docker ps -a                        # Voir tous les conteneurs
docker stop [nom]                   # Arrêter
docker rm [nom]                     # Supprimer
docker logs [nom]                   # Voir les logs
docker-compose up -d                # Lancer avec compose
docker-compose down                 # Arrêter avec compose
docker-compose ps                   # État des services
docker-compose logs                 # Logs des services
```

- Commandes relatives à la manipulation des images, notamment :
    - Lister les images :docker images
    - Télécharger une image :docker pull <image>
    - Supprimer une image :docker rmi <image>
- Commandes relatives à la manipulation des conteneurs
    - Supprimer le conteneur docker :docker rm <id>
    - Voir si le conteneur est démarré :docker ps
    - Affiche tout les conteneurs :docker ps -a

- Les commandes relatives à la manipulation des volumes, notamment :
    - Lister les volumes : docker volume ls
    - Créer un volume : docker volume create <volume_name>
    - Supprimer un volume : docker volume rm <volume_name>
- Les commandes relatives à la manipulation des networks, notamment:
    - Lister les networks : docker network ls
    - Créer un network : docker network create <network_name>
- Supprimer les networks : docker network rm <network_name>
- Comment télécharger une image avec la commande : docker pull <image>
- Comment lancer un conteneur en utilisant l'image téléchargée :
    - En mode interactif avec la commande : docker run -it <image>
    - En tâche de fond avec la commande : docker run -d <image>
    - Avec un volume avec la commande : docker run -v <volume_name>:/path/to/dir -it <image>
    - En exposant des ports du conteneur avec la commande : docker run -p <host_port>:<container_port> -it <image>

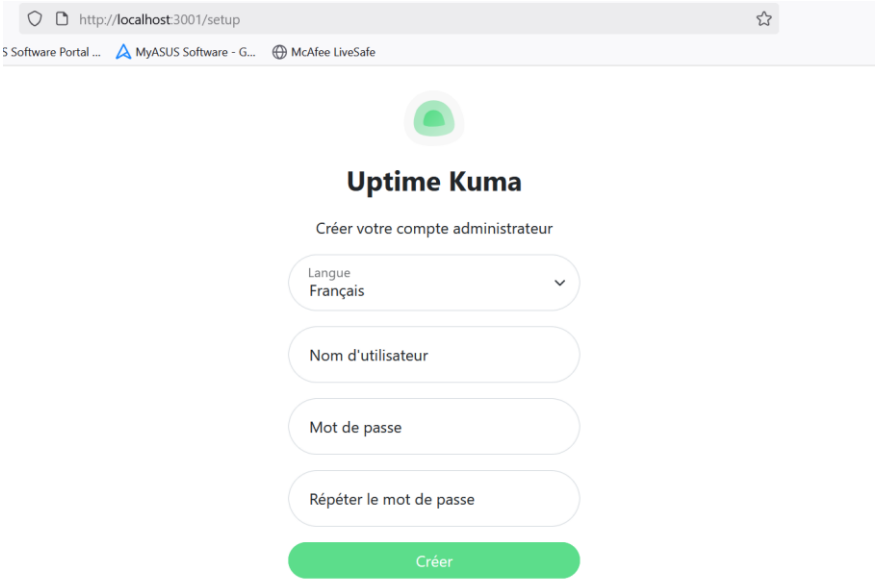
Jour 3-5 : Uptime Kuma – Monitoring de disponibilité ° Objectif

```
theot@Asus04 MINGW64 ~
$ cd Desktop

theot@Asus04 MINGW64 ~/Desktop
$ mkdir uptime-kuma

theot@Asus04 MINGW64 ~/Desktop
$ nano docker-compose.yml

theot@Asus04 MINGW64 ~/Desktop
$ docker-compose up -d
[+] Running 13/13
 ✔ uptime-kuma Pulled                                          57.3s
   ✔ b338562f40a7 Pull complete                                 6.8s
   ✔ 874bf4d93720 Pull complete                                 6.9s
   ✔ b16337721583 Pull complete                                 9.6s
   ✔ 7d955db85b85 Pull complete                                 9.8s
   ✔ 2c706596bd17 Pull complete                                 9.9s
   ✔ 88a5c59ed14f Pull complete                                 9.9s
   ✔ 5a1d0a896c33 Pull complete                                14.0s
   ✔ e68c2f25b946 Pull complete                                14.4s
   ✔ 2e6c90f010d6 Pull complete                                14.5s
   ✔ ff15b10fabb8 Pull complete                                14.6s
   ✔ 4f4fb700ef54 Pull complete                                14.6s
   ✔ d2a400cc8adb Pull complete                                54.8s
[+] Running 2/2
 ✔ Network desktop_default  Created                             0.8s
 ✔ Container uptime-kuma    Started                            26.2s

theot@Asus04 MINGW64 ~/Desktop
$ |
```

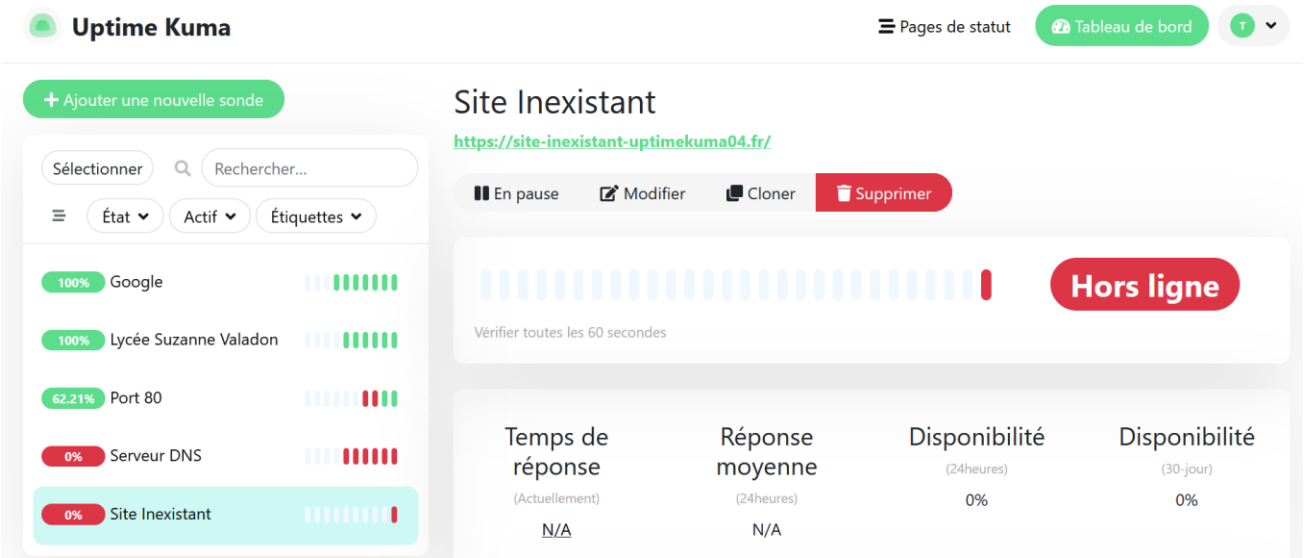
```
  GNU nano 8.3                          docker-compose.yml
services:
  uptime-kuma:
    image: louislam/uptime-kuma:1
    container_name: uptime-kuma
    volumes:
      - ./data:/app/data
    ports:
      - "3001:3001"
    restart: unless-stopped
```

Explication :

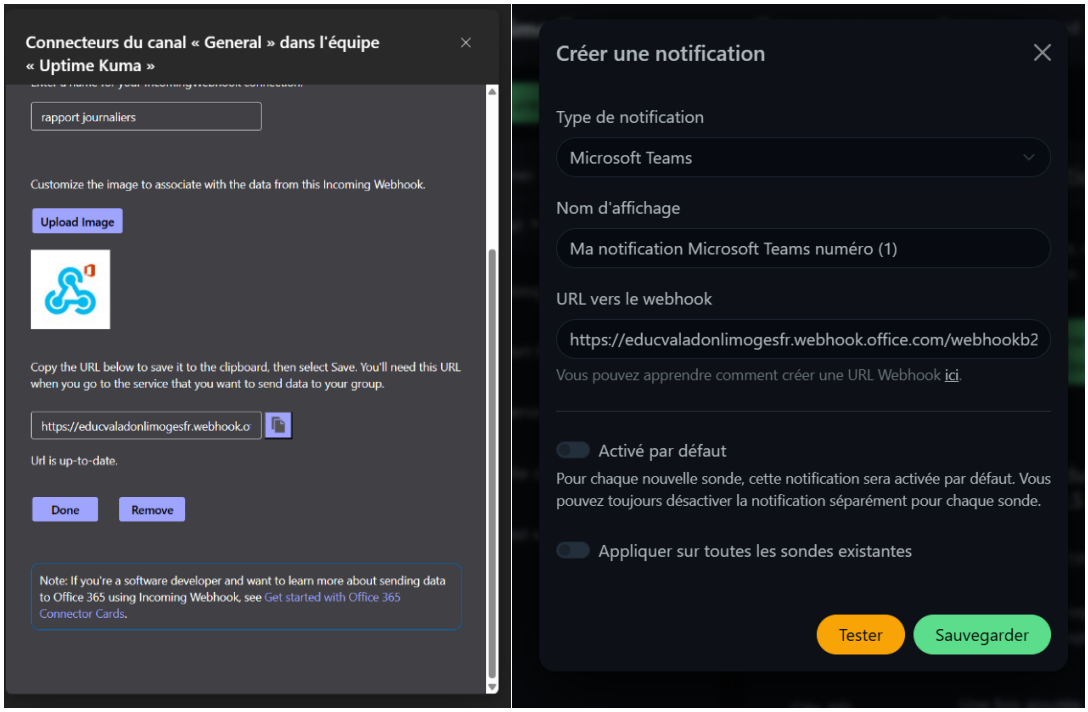
- **cd Desktop :** se déplace dans le 'Bureau'
- **mkdir uptim-kuma :** créer un dossier 'uptime-kuma' dans 'Desktop'
- **nano docker-compose.yml :** édite le fichier « docker-compose.yml »
- **docker-compose up -d :** lance le tout



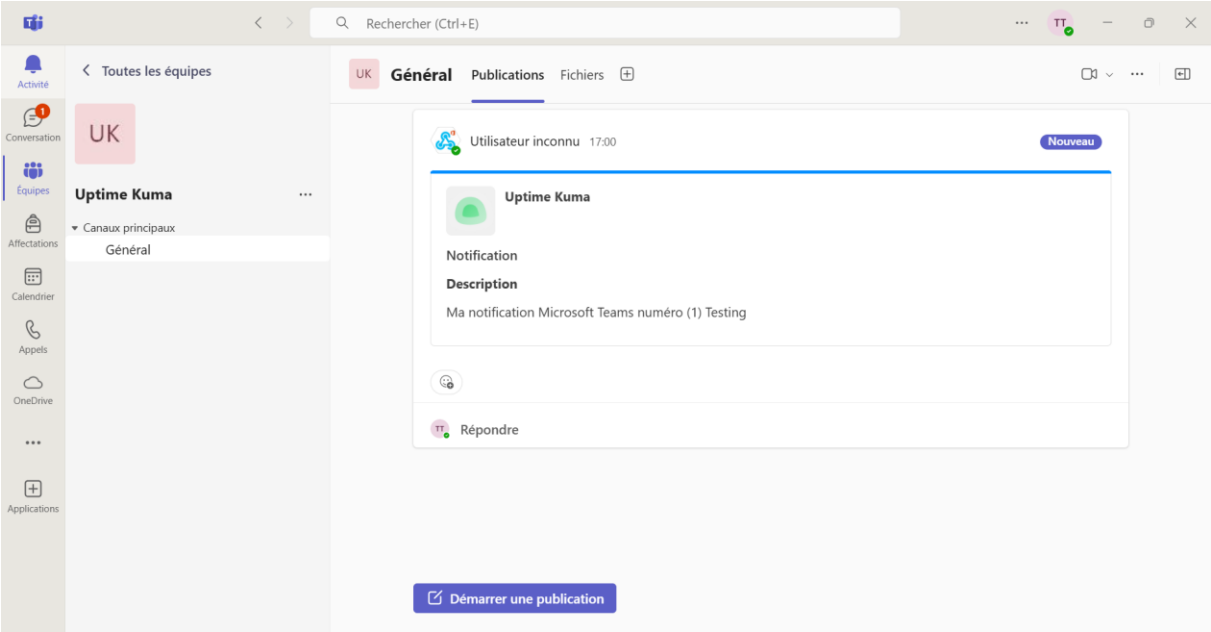
- **Configuration de base :**
  - Créer un compte admin (NOTER LE MDP !) :
    id :theo.trn ; mdp :stage004*
  - Ajouter 5 monitors :
    - Google(HTTP)
    - Site de l'école (HTTP)
    - Un Serveur DNS (8.8.8.8)
    - Un Port (TCP port 80 sur google.com)
    - Un site qui n'existe pas (pour voir les erreurs)



  - Configurer une notification (email ou Discord) :
    Création d'un canal Teams pour les notifications :
    Création d'un Webhook sur Teams pour les notifications :



Notification reçu :



Semaine 2 : Zabbix – Monitoring professionnel

Jour 6-7 : Comprendre Zabbix

- **A quoi ça sert ? :**

Zabbix est un logiciel qui supervise de nombreux paramètres réseaux ainsi que la santé et l'intégrité des serveurs. Il utilise un mécanisme de notification flexible qui permet aux utilisateurs de configurer une base d'alerte e-mail pour pratiquement tous les événements. Cela permet une réponse rapide aux problèmes serveurs. De plus, il offre un excellent reporting (communication de données) et des fonctionnalités de visualisation de données basées sur les données stockées.

Zabbix supporte à la fois "polling et trapping". Tous les rapports et statistiques, comme la configuration de paramètres, sont accessibles par l'interface web. Cette interface web veille à ce que le statut du réseau et des serveurs puisse être évalué depuis n'importe quel endroit. Zabbix peux jouer un rôle important dans la supervision de l'infrastructure IT. Surtout pour les petites organisations avec peu de serveurs ainsi que pour les grandes entreprises avec une multitude de serveurs.

Zabbix est gratuit et est écrit et distribué sous Licence publique générale GNU version 2. Cela signifie que son code source est librement distribué et disponible pour le public. Le support gratuit et commercial est disponible et fourni par Zabbix Company.

- **Surveiller des serveurs complets :**
  Le fonctionnement de Zabbix repose sur un serveur et un agent et il réagit par rapport à des événements. La supervision et métrologie Zabbix peut superviser soit des services système, des services applicatifs en relation à des applications métiers, de la disponibilité de service, etc.. Et tout cela se trouve dans des templates. On attribue ces templates à des hôtes suivant ce que l'on souhaite superviser.
  On peut également créer ses propres templates, surtout lorsqu'on supervise des applications métiers spécifiques. Ou bien pour la supervision d'infrastructure bien spécifique (matériels spécifiques, règles de pare-feu, etc…)

- **CPU, RAM, disque, réseau :**
  ZABBIX requiert à la fois de la mémoire physique et de la mémoire disque. 128 MB de mémoire physique et 256 MB d'espace disque libre peuvent être suffisant. Cependant, la valeur de la mémoire disque requise dépend évidemment du nombre d'hôtes ainsi que des paramètres qui seront supervisé.
  La base de données ZABBIX peut nécessiter d'importantes ressources CPU en fonction du nombre de paramètres supervisés ainsi que du moteur de base de données choisis.
  Il supporte de nombreuses plateformes tels que Linux, Mac OS/X, OpenBSD , Solaris, Windows…

- **Alertes automatiques :**
  Avec des éléments collectant des données et des déclencheurs conçus pour «déclencher» des situations problématiques, il est donc intéressant de disposer d'un mécanisme d'alerte qui informe des événements importants même si l'interface de Zabbix n'est pas consulté. Voici une documentation complète sur : comment mettre en place l'alerte automatique :
  https://www.zabbix.com/documentation/5.2/fr/manual/quickstart/notification

- **Utilisé par les grandes entreprises**
  De par ses nombreuses compétences, Zabbix est donc utilisé par de nombreuses entreprises tels que : OpenText, Leihia, TIVIT, Global Hitss…

Jour 8-10 : Déploiement Zabbix simplifié

- **Installation via Docker Compose :**

```
theot@Asus04 MINGW64 ~
$ cd Desktop

theot@Asus04 MINGW64 ~/Desktop
$ mkdir zabbix

theot@Asus04 MINGW64 ~/Desktop
$ cd zabbix

theot@Asus04 MINGW64 ~/Desktop/zabbix
$ nano docker-compose.yml

theot@Asus04 MINGW64 ~/Desktop/zabbix
$ |
```

cd Desktop: se positionne dans le Desktop (Bureau)

mkdir zabbix : créer un dossier « zabbix »

cd zabbix : se positionne dans « zabbix »

nano docker-compose.yml : créer un fichier docker-compose.yml

docker-compose ps : permet de vérifier qu'il n'y a pas d'erreur

```
  GNU nano 8.3                                                    docker-compose.yml
version: '3.8'

services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root_pwd
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix_pwd
    volumes:
      - mysql-data:/var/lib/mysql
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_bin

  zabbix-server:
    image: zabbix/zabbix-server-mysql:alpine-6.0-latest
    ports:
      - "10051:10051"
    environment:
      DB_SERVER_HOST: mysql
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix_pwd
      MYSQL_ROOT_PASSWORD: root_pwd
    depends_on:
      - mysql
    volumes:
      - zabbix-data:/var/lib/zabbix

  zabbix-web:
    image: zabbix/zabbix-web-nginx-mysql:alpine-6.0-latest
    ports:
      - "8080:8080"
    environment:
      ZBX_SERVER_HOST: zabbix-server
      DB_SERVER_HOST: mysql
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix_pwd
      MYSQL_ROOT_PASSWORD: root_pwd
      PHP_TZ: Europe/Paris
    depends_on:
      - mysql
      - zabbix-server
                                                                  [ Read 57 lines ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Lin
```

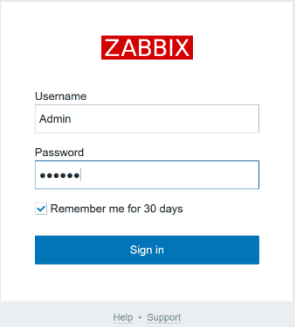
```
theot@Asus04 MINGW64 ~/Desktop/zabbix
$ docker-compose up -d
time="2025-05-28T11:56:32+02:00" level=warning msg="C:\\Users\\theot\\Desktop\\zabbi
x\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, plea
se remove it to avoid potential confusion"
[+] Running 32/32
 ✔ zabbix-agent Pulled                                                    110.1s
   ✔ 6a39e38ff334 Pull complete                                            97.8s
   ✔ 0f798d28c0de Pull complete                                            98.7s
   ✔ 908596c835cc Pull complete                                            99.3s
   ✔ 7548e2ad241f Pull complete                                           104.2s
   ✔ 6142408563 40 Pull complete                                          105.0s
 ✔ mysql Pulled                                                           131.1s
   ✔ c2eb5d06bfea Pull complete                                            29.8s
   ✔ 253ce0d09858 Pull complete                                            29.8s
   ✔ fb10e0b9f49b Pull complete                                            30.0s
   ✔ 142bf0fa8655 Pull complete                                            30.5s
   ✔ 8609c4bc4339 Pull complete                                            30.6s
   ✔ a48ef6c0e15d Pull complete                                            30.7s
   ✔ 26b53dc04f70 Pull complete                                            48.9s
   ✔ cc6ed31dcc2a Pull complete                                            49.0s
   ✔ e0211b440535 Pull complete                                           126.1s
   ✔ 6c1e029c2325 Pull complete                                           126.2s
   ✔ c0468773f197 Pull complete                                           126.2s
 ✔ zabbix-server Pulled                                                   113.5s
   ✔ 97081e440948 Pull complete                                            84.1s
   ✔ f6e51f703503 Pull complete                                            84.9s
   ✔ 1f087732637d Pull complete                                            85.7s
   ✔ c155b3e30b3b Pull complete                                            88.9s
   ✔ 446ad3d9856d Pull complete                                           108.2s
   ✔ b71f5ca4958c Pull complete                                           108.4s
 ✔ zabbix-web Pulled                                                      104.9s
   ✔ f18232174bc9 Already exists                                            0.0s
   ✔ 793b0d6426d8 Pull complete                                            85.0s
   ✔ ed06a5afbfa9 Pull complete                                            85.1s
   ✔ a93678093d96 Pull complete                                            99.7s
   ✔ 4f4fb700ef54 Pull complete                                           108.4s
   ✔ 99f555ae719d Pull complete                                            99.9s
[+] Running 7/7
 ✔ Network zabbix_default            Created                               0.2s
 ✔ Volume "zabbix_mysql-data"        Created                               0.0s
 ✔ Volume "zabbix_zabbix-data"       Created                               0.0s
 ✔ Container zabbix-mysql-1          Started                               2.5s
 ✔ Container zabbix-zabbix-server-1  Sta...                                2.1s
 ✔ Container zabbix-zabbix-agent-1   Star...                               2.6s
 ✔ Container zabbix-zabbix-web-1     Starte...                             2.7s
```

```
theot@Asus04 MINGW64 ~/Desktop/zabbix
$ docker-compose ps
time="2025-05-28T12:00:36+02:00" level=warning msg="C:\\Users\\theot\\Desktop\\zabbix\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please r
emove it to avoid potential confusion"
NAME                    IMAGE                                        COMMAND                  SERVICE         CREATED              STATUS
    PORTS
zabbix-mysql-1          mysql:8.0                                    "docker-entrypoint.s…"   mysql           About a minute ago   Up About a minute
    3306/tcp, 33060/tcp
zabbix-zabbix-agent-1   zabbix/zabbix-agent:alpine-6.0-latest        "/sbin/tini -- /usr/…"   zabbix-agent    About a minute ago   Up About a minute
    10050/tcp
zabbix-zabbix-server-1  zabbix/zabbix-server-mysql:alpine-6.0-latest "/sbin/tini -- /usr/…"   zabbix-server   About a minute ago   Up About a minute
    0.0.0.0:10051->10051/tcp
zabbix-zabbix-web-1     zabbix/zabbix-web-nginx-mysql:alpine-6.0-latest "docker-entrypoint.sh" zabbix-web      About a minute ago   Up About a minute (health: starting
)   0.0.0.0:8080->8080/tcp, 8443/tcp
```

Création du docker-compose.yml avec toutes ses règles puis envoie du fichier grâce à « up -d »

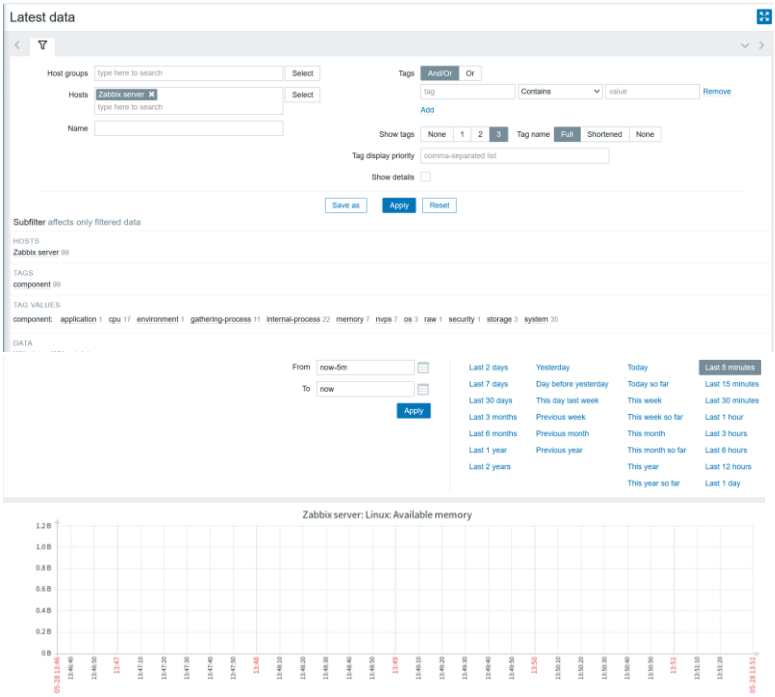
- **Utilisation basique:**

  - **Se connecter à l'interface :**



  - **Explorer le dashboard :**

  - **Voir les données du "Zabbix server" :**



  - **Comprendre : Hosts, Items, Triggers, Graphs :**

    Hoste :

    -tout dispositif physique ou virtuel, toute application, service ou tout autre ensemble logiquement lié à des paramètres surveillés.

    Items :

-une donnée particulière que l’on souhaite recevoir d'un hôte, une métrique de données.

Triggers :

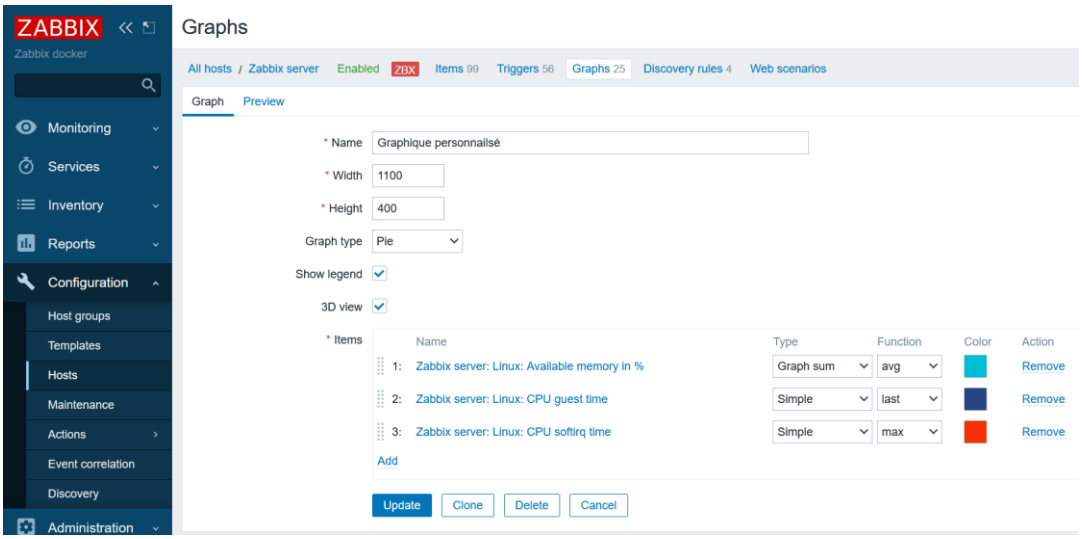
- une expression logique qui définit un seuil de problème et est utilisée pour "évaluer" les données reçues dans les articles.

Lorsque les données reçues sont supérieures au seuil, les déclencheurs passent de 'Ok' à un état 'Problème'. Lorsque les données reçues sont inférieures au seuil, les déclencheurs restent/retours à un état «Ok».

Graphs :

-consultables à partir de la surveillance et des hôtes en cliquant sur Graphs pour l'hôte concerné. Tout graphe personnalisé qui a été configuré pour l'hôte peut être affiché, ainsi que n'importe quel graphe simple.

- **Créer un graphique personnalisé :**



- **Mini-projet : Créer une alerte si CPU > 80%**

Semaine 3: Logs et Visualisation

Jour 11-12 : Dozzle – Logs et Visualisation :

- **Pourquoi Dozzle ? :**

  Dozzle est plus simple à prendre en main que Loki. Il est possible de déployer et dimensionner des applications dans n’importe quel environnement, avec l’assurance que le code s’exécutera correctement.

- **Installation :**

  Création du dossier dozzle → mkdir dozzle

  Se déplacer dans le dossier → cd dozzle

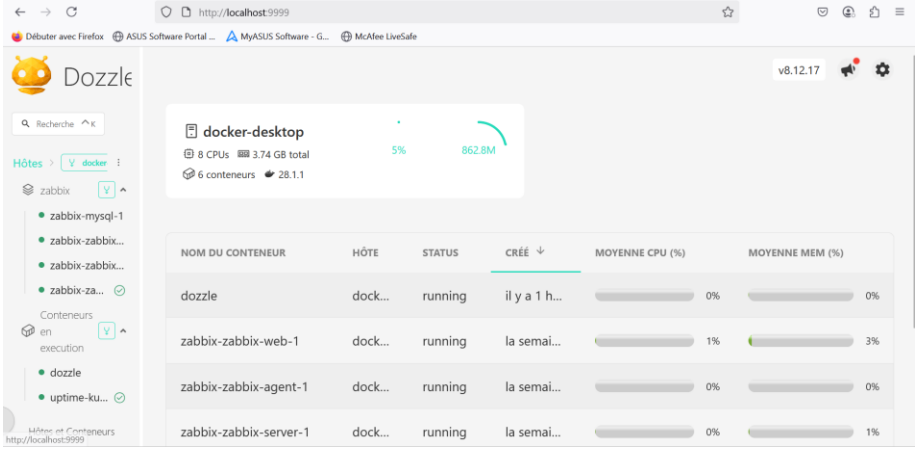
  Créer un fichier docker-compose.yml → nano docker-compose.yml

  Rajouter le code suivant :

```
  GNU nano 8.3                                                    docker-compose.yml
services:
  dozzle:
    container_name: dozzle
    image: amir20/dozzle:latest
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - /c/Users/theot/desktop/dozzle/users.yaml:/home/app/users.yaml:ro

    ports:
      - "9999:8080"
    environment:
      DOZZLE_ENABLE_ACTIONS: "true"
```

Lancer dozzle → docker-compose.yml up -d

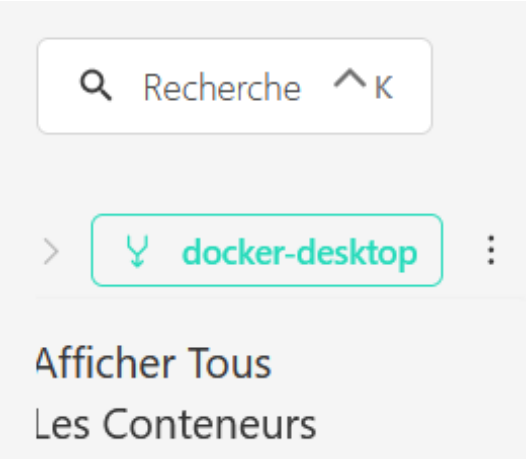
Se rendre sur le localhost avec le port renseigné (ici 9999) :



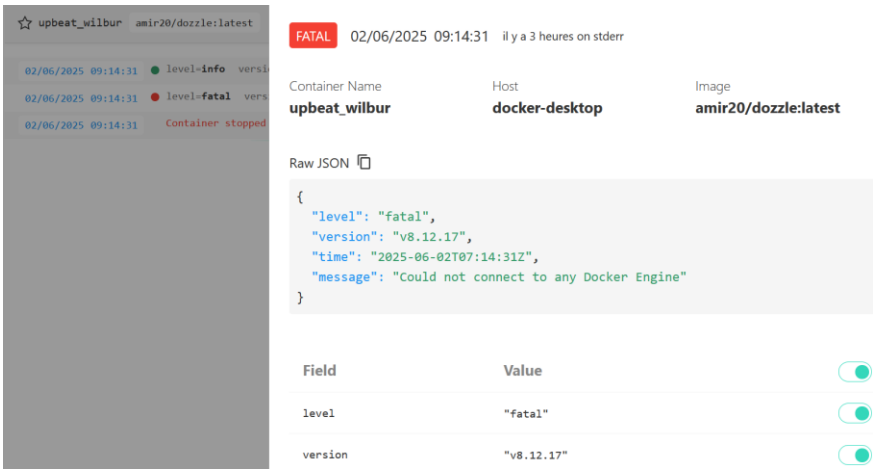
- **Utilisation :**

  - **Voir tous les logs en temps réel :**

  - **Filtrer par conteneurs :**



- **Rechercher dans les logs et Comprendre les erreurs :**



Jour 13-15 : Grafana – Tableau de bord :

- **Installation:**

  Création du dossier grafana → mkdir grafana

  Se déplacer dans le dossier → cd grafana

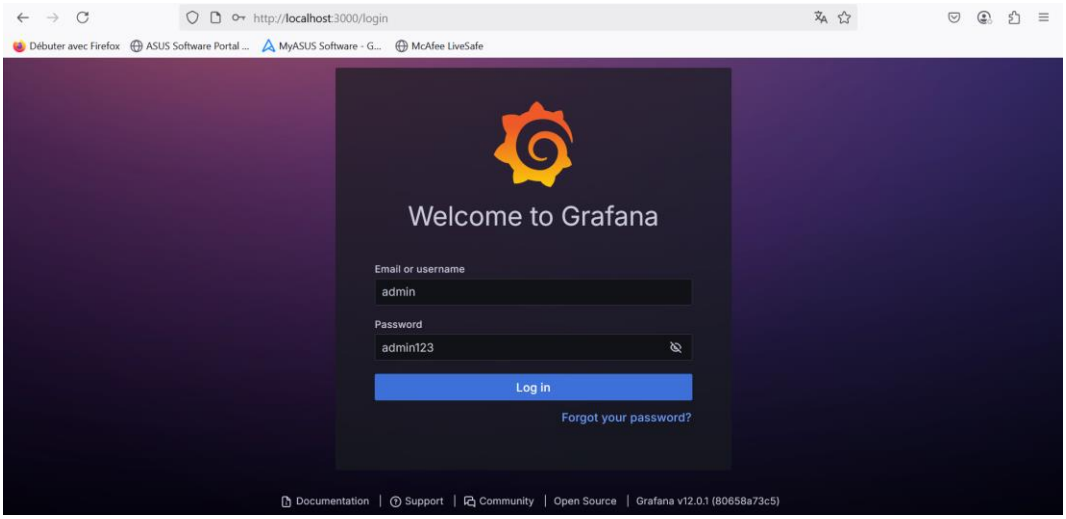
  Créer un fichier docker-compose.yml → nano docker-compose.yml

  Rajouter le code suivant :

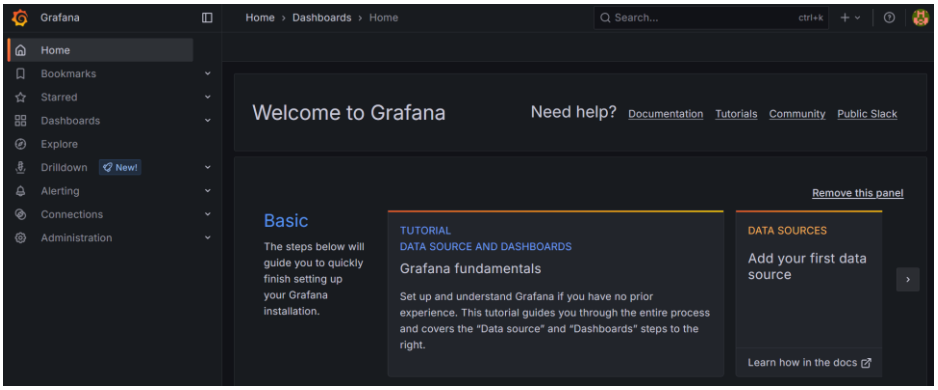
```
  GNU nano 8.3                                                docker-compose.yml
version: '3.8'
services:
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin123
      - GF_INSTALL_PLUGINS=
    volumes:
      - grafana-data:/var/lib/grafana
    restart: unless-stopped

volumes:
  grafana-data:
```

  Lancer grafana → docker-compose.yml up -d

  Se rendre sur le localhost avec le port renseigné (ici 3000) :



- **Se connecter (admin/admin123) :**



- **Ajouter Zabbix comme data source :**

  Pour cela, il faut trouver le plugin officiel développé par Alxander Zobnin qui est le seul plugin Grafana maintenu et reconnu pour connecter Zabbix comme source de données (listé officiellement sur le site de Grafana). Il faudrait donc le rajouter dans le docker-compose.yml de Grafana. Cependant, depuis Grafana 8+, les plugins doivent être signés. Pour qu'un plugin communautaire fonctionne, il faut activer explicitement son autorisation. Au lieu d'avoir :

```
      - GF_INSTALL_PLUGINS=alexanderzobnin-zabbix-datasource
```

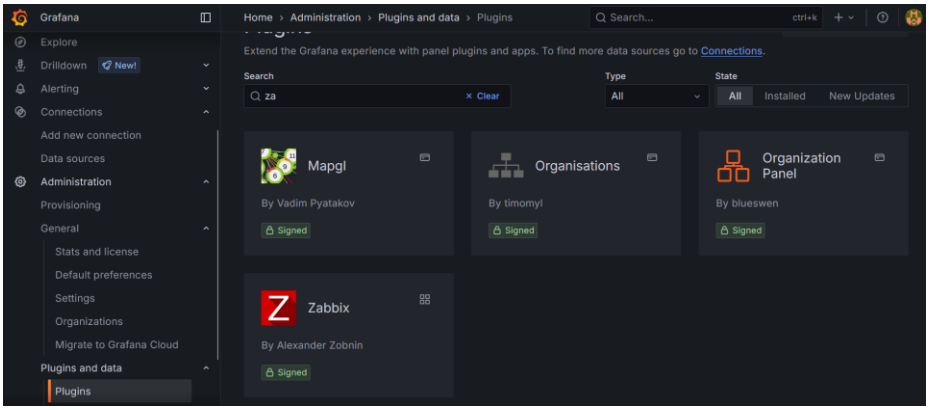
  Il faudra réécrire le docker-compose.yml :

```
  GNU nano 8.3                                          docker-compose.yml
version: '3.8'
services:
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin123
      - GF_PLUGINS_ALLOW_LOADING_UNSIGNED_PLUGINS=alexanderzobnin-zabbix-datasource
    volumes:
      - grafana-data:/var/lib/grafana
    restart: unless-stopped
    command:
      - bash
      - -c
      - |
        grafana-cli plugins install alexanderzobnin-zabbix-datasource && \
        /run.sh

volumes:
  grafana-data:
```

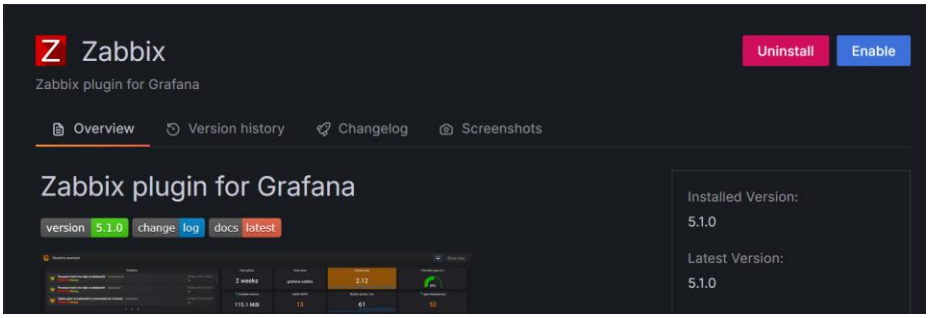
  GF_PLUGINS_ALLOW_LOADING_UNSIGNED_PLUGINS : autorise ce plugin non signé (nécessaire en 2025).

  Le **command:** exécute le grafana-cli avant de lancer le serveur (/run.sh est le script d'entrée de l'image officielle).

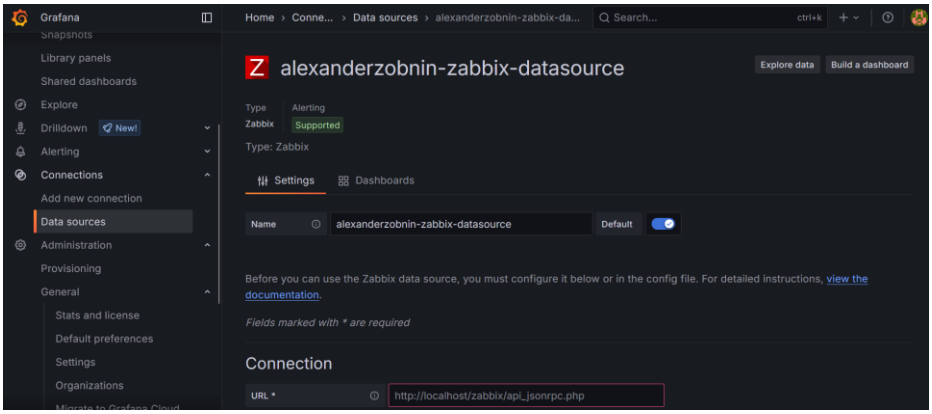
  Une fois fait, il faut se rendre dans « Administration », « Général », « Plugins »



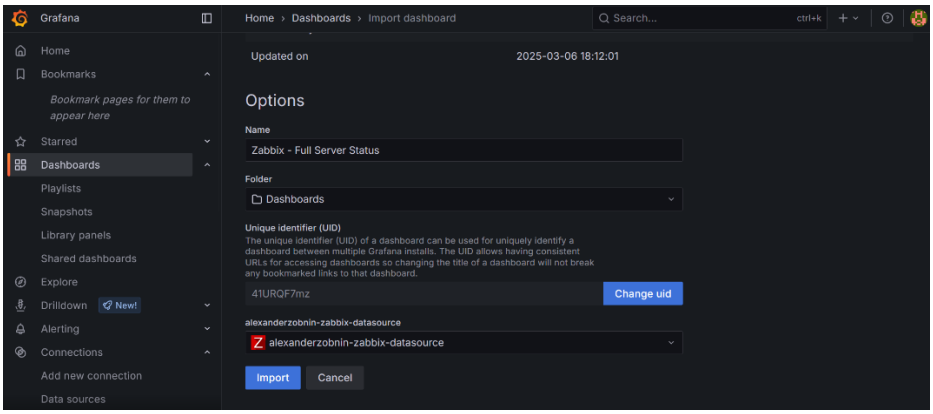
Puis l'installer et l'activer:

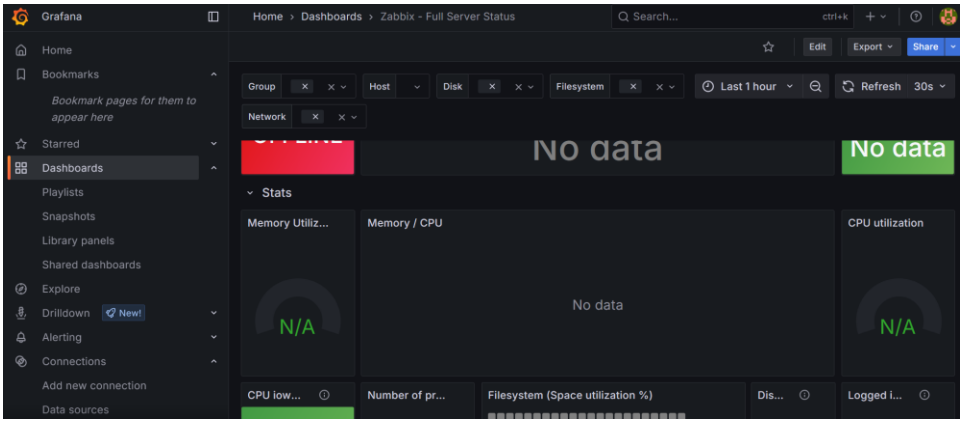


- **Ajouter Zabbix comme data source :**

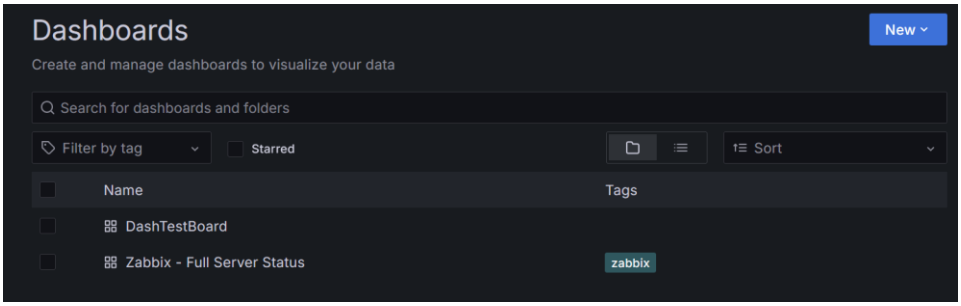


- **Importer un dashboard Zabbix (ID : 5363) :**





Créer un dashboard + Explorer les visualisation :



Semaine 4: Projet d'intégration et sécurité basique

Jour 16-17 : Wazuh:

- **Installation:**

```
theot@Asus04 MINGW64 ~/desktop/wazuh
$ git clone https://github.com/wazuh/wazuh-docker.git -b v4.12.0
Cloning into 'wazuh-docker'...
remote: Enumerating objects: 15054, done.
remote: Counting objects: 100% (142/142), done.
remote: Compressing objects: 100% (80/80), done.
remote: Total 15054 (delta 88), reused 72 (delta 62), pack-reused 14912 (from 3)
Receiving objects: 100% (15054/15054), 315.13 MiB | 15.39 MiB/s, done.
Resolving deltas: 100% (7876/7876), done.
warning: refs/tags/v4.12.0 0092a509031b2c5a4be79501d9b93c0c12d18b21 is not a commit!
Note: switching to '0f15acb78377a9b240ebd12e1b46910eb54055b6'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false
```

Avec la commande **git clone https://github.com/wazuh/wazuh-docker.git -b v4.12.0,** nous récupérons la dernière version via le lien en précisant **-b** et **.git** pour mettre à jour la commande.

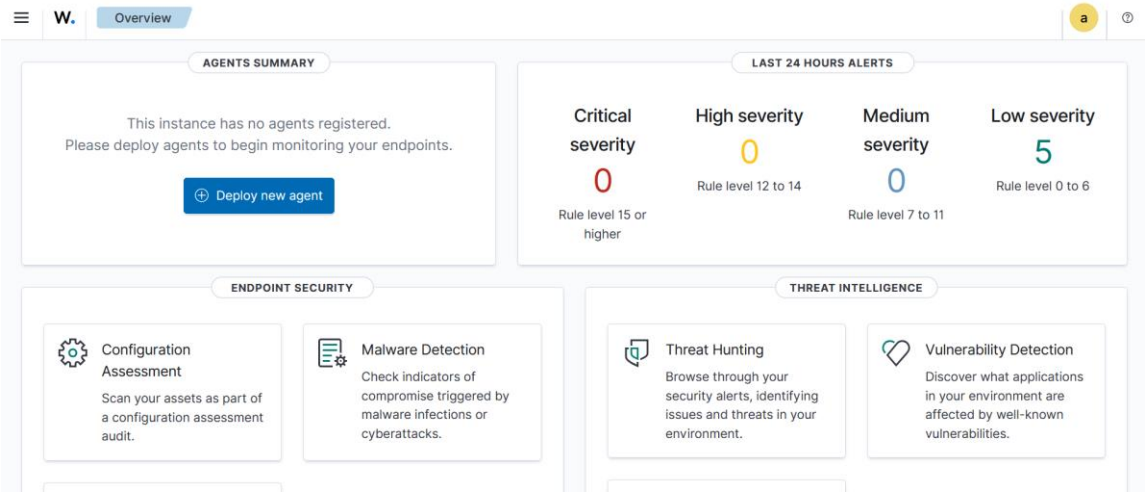
```
theot@Asus04 MINGW64 ~/desktop/wazuh
$ ls
wazuh-docker/

theot@Asus04 MINGW64 ~/desktop/wazuh
$ cd wazuh-docker/

theot@Asus04 MINGW64 ~/desktop/wazuh/wazuh-docker ((v4.12.0))
$ ls
CHANGELOG.md  SECURITY.md          indexer-certs-creator/
LICENSE       VERSION.json         multi-node/
README.md     build-docker-images/ single-node/

theot@Asus04 MINGW64 ~/desktop/wazuh/wazuh-docker ((v4.12.0))
$ ls -al
total 91
drwxr-xr-x 1 theot 197609     0 Jun  3 15:42 ./
drwxr-xr-x 1 theot 197609     0 Jun  3 15:42 ../
-rw-r--r-- 1 theot 197609   175 Jun  3 15:42 .env
drwxr-xr-x 1 theot 197609     0 Jun  3 15:42 .git/
drwxr-xr-x 1 theot 197609     0 Jun  3 15:42 .github/
-rw-r--r-- 1 theot 197609   196 Jun  3 15:42 .gitignore
-rw-r--r-- 1 theot 197609 19841 Jun  3 15:42 CHANGELOG.md
-rw-r--r-- 1 theot 197609 25211 Jun  3 15:42 LICENSE
-rw-r--r-- 1 theot 197609 11895 Jun  3 15:42 README.md
-rw-r--r-- 1 theot 197609  3669 Jun  3 15:42 SECURITY.md
-rw-r--r-- 1 theot 197609    52 Jun  3 15:42 VERSION.json
drwxr-xr-x 1 theot 197609     0 Jun  3 15:42 build-docker-images/
drwxr-xr-x 1 theot 197609     0 Jun  3 15:42 indexer-certs-creator/
drwxr-xr-x 1 theot 197609     0 Jun  3 15:42 multi-node/
drwxr-xr-x 1 theot 197609     0 Jun  3 15:42 single-node/

theot@Asus04 MINGW64 ~/desktop/wazuh/wazuh-docker ((v4.12.0))
$ cd single-node/

theot@Asus04 MINGW64 ~/desktop/wazuh/wazuh-docker/single-node ((v4.12.0))
$ ls -al
total 25
drwxr-xr-x 1 theot 197609    0 Jun  3 15:42 ./
drwxr-xr-x 1 theot 197609    0 Jun  3 15:42 ../
-rw-r--r-- 1 theot 197609  892 Jun  3 15:42 README.md
drwxr-xr-x 1 theot 197609    0 Jun  3 15:42 config/
-rw-r--r-- 1 theot 197609 4396 Jun  3 15:42 docker-compose.yml
-rw-r--r-- 1 theot 197609  302 Jun  3 15:42 generate-indexer-certs.yml
```

```
theot@Asus04 MINGW64 ~/desktop/wazuh/wazuh-docker/single-node ((v4.12.0))
$ docker compose -f generate-indexer-certs.yml up
time="2025-06-03T15:50:32+02:00" level=warning msg="C:\\Users\\theot\\desktop\\wazuh
\\wazuh-docker\\single-node\\generate-indexer-certs.yml: the attribute `version` is
obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 5/5
 ✔ generator Pulled                                                          10.9s
   ✔ 17d0386c2fff Pull complete                                               7.6s
   ✔ 7ce91ec7d1d3 Pull complete                                               8.5s
   ✔ 5249716d429c Pull complete                                               8.5s
   ✔ d7003467fd14 Pull complete                                               8.6s
[+] Running 2/2
 ✔ Network single-node_default        Created                                0.2s
 ✔ Container single-node-generator-1  C...                                   0.5s
Attaching to generator-1
generator-1  | The tool to create the certificates exists in the in Packages bucket
generator-1  | 03/06/2025 13:50:46 INFO: Generating the root certificate.
generator-1  | 03/06/2025 13:50:46 INFO: Generating Admin certificates.
generator-1  | 03/06/2025 13:50:46 INFO: Admin certificates created.
generator-1  | 03/06/2025 13:50:46 INFO: Generating Wazuh indexer certificates.
generator-1  | 03/06/2025 13:50:46 INFO: Wazuh indexer certificates created.
generator-1  | 03/06/2025 13:50:46 INFO: Generating Filebeat certificates.
generator-1  | 03/06/2025 13:50:46 INFO: Wazuh Filebeat certificates created.
generator-1  | 03/06/2025 13:50:46 INFO: Generating Wazuh dashboard certificates.
generator-1  | 03/06/2025 13:50:46 INFO: Wazuh dashboard certificates created.
generator-1  | Moving created certificates to the destination directory
generator-1  | Changing certificate permissions
generator-1  | Setting UID indexer and dashboard
generator-1  | Setting UID for wazuh manager and worker
generator-1 exited with code 0

w View in Docker Desktop   o View Config   w Enable Watch
```

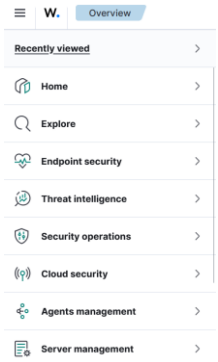
Grâce à : **docker compose -f generate-indexer-certs.yml up**, nous générons les certifications pour garantir l'authentification, le chiffrement et l'intégrité.

```
theot@Asus04 MINGW64 ~/desktop/wazuh/wazuh-docker/single-node ((v4.12.0))
$ docker compose -f docker-compose.yml up -d
time="2025-06-04T10:14:50+02:00" level=warning msg="Found orphan containers ([single
-node-generator-1]) for this project. If you removed or renamed this service in your
 compose file, you can run this command with the --remove-orphans flag to clean it u
p."
[+] Running 3/3
 ✔ Container single-node-wazuh.manager-1    Started                              0.6s
 ✔ Container single-node-wazuh.indexer-1    Started                              0.5s
 ✔ Container single-node-wazuh.dashboard-1  Started                              1.0s
```

Lancement des conteneurs et se connecter au https://localhost/ puis admin – SecretPassword



Explorer l’interface:



Voir les événements de sécurité:

Gestion des événements de sécurité (Security Information managment) La catégorie “ Security Information Management “ de la plateforme Wazuh simplifie la gestion des informations de sécurité en permettant de parcourir facilement les alertes, tout en identifiant de manière proactive les problèmes et les menaces potentielles dans l’environnement informatique. Cette fonctionnalité offre une visibilité approfondie, permettant une réponse rapide aux incidents de sécurité.

Comprendre le principe du SIEM:

Un élément essentiel d'une cybersécurité efficace est une solution de gestion des informations et des événements de sécurité (SIEM). Ces types de solutions collectent, agrègent et analysent de grands volumes de données provenant d'applications, de dispositifs, de serveurs et d'utilisateurs à l'échelle de l'organisation en temps réel. En consolidant cette vaste gamme de données sur une plateforme unique et unifiée, les solutions SIEM offrent une vue d'ensemble de la posture de sécurité d'une entreprise, permettant aux centres d'opérations de sécurité (SOC) de détecter, d'enquêter et de répondre rapidement et efficacement aux incidents de sécurité. Les solutions SIEM peuvent aider les organisations de toutes tailles :

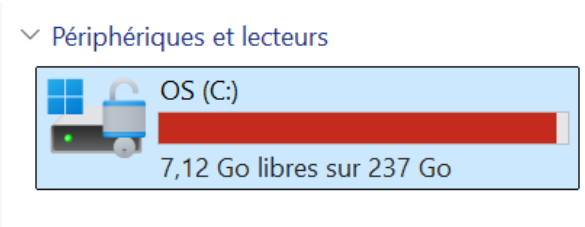
- Gagnez en visibilité sur l’état de leur sécurité en centralisant et en analysant les données provenant de sources disparates.
- Détectez et identifiez les failles de sécurité et menaces potentielles en temps réel, ce qui réduit le risque de compromission.
- Examinez et triez efficacement les incidents de sécurité, ce qui réduit le temps et les ressources nécessaires à leur résolution.
- Conformez-vous aux normes et cadres de sécurité réglementaires et spécifiques au secteur.

WAZUH est une plateforme de sécurité SIEM open source en combinant les fonctionnalités de collecte, de centralisation et de corrélation des événements de sécurité, la détection des menaces, la surveillance de l'intégrité, la réponse aux incidents et la conformité. Basée sur une

architecture client-serveur, WAZUH offre une visibilité 360 degré sur l'infrastructure informatique, des postes de travail aux serveurs cloud.

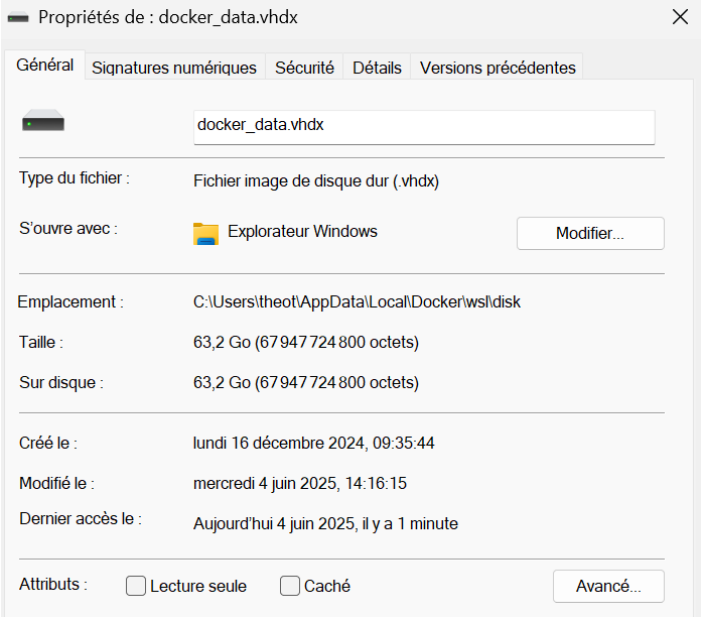
Suppression de « docker_data.vdhx » :

Au file des missions, je me suis rendu compte que mon espace de stockage diminué considérablement. A tel point que je suis, à un moment, tombé à 200 mo de stockage libre.



Apres avoir chercher dans de nombreux dossier, j'ai remarque un fichier qui prenait beaucoup plus que les autres : docker_data.vhdx.

En effet, à lui seul il prenait plus de 60GO :

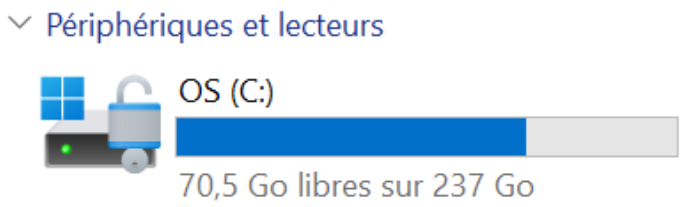


La solution simple et « naturelle » serait de le supprimer. Cependant, il contient tous les conteneurs, les images, les volumes, les logs… le supprimer serait donc perdre tout ça.

Alors la solution trouvé et de compacter le tout, de tel sorte à supprimer uniquement les données non-utilisées. Cependant, pour y arrivé, il faut utiliser Hyper-V

(bien qu'avant il faut réussir à supprimer les conteneurs/volumes souhaités avec un Docker qui ne souhaite pas se lancer + le WSL qui ne souhaite pas se lancer (wsl –shutdown)… . **Hyper-V** est la technologie de **virtualisation** de Microsoft. Elle permet à Windows de créer et gérer des **machines virtuelles (VM)**, un peu comme VirtualBox ou VMware. **Docker Desktop** sous Windows utilise souvent **WSL 2** *et* **Hyper-V** en arrière-plan pour fonctionner plus efficacement, surtout pour stocker ses données dans des disques virtuels comme les .vhdx. Cependant, Hyper-V n'est pas disponible sur Windows Family. Il m'est donc actuellement impossible de compacter se fichier .vhdx contenait des dizaines de giga octets de logs ne servant à rien. Apres de nombreuse tentative, j'ai décidé de supprimer le fichier et relancer tout les conteneurs un à un.

Wazuh étant le principale acteur de ces démarches, a donc était supprimé.



Mes erreurs ont été :

1. **Pas de nettoyage régulier de Docker**
   Docker stocke beaucoup de données : images, conteneurs, volumes, réseaux, logs… Sans nettoyage, ça grossit vite, surtout avec des outils lourds comme Wazuh.
2. **Volumes Docker non nettoyés**
   Les volumes persistent même après la suppression des conteneurs. Ils peuvent prendre énormément de place (logs, bases de données, données persistantes).
3. **Pas de surveillance de l'espace disque**
   Pas de vérification régulière pour voir où va l'espace utilisé par Docker (via docker system df, docker volume ls, etc).
4. **Manque de sauvegarde / export des données importantes**
   En cas de problème, je dois repartir de zéro parce que je n'ais pas sauvegardé les volumes ou fichiers importants.
5. **Manque de connaissances sur WSL + Docker sur Windows**
   La couche WSL, les fichiers VHDX, les mécanismes de stockage, peuvent bloquer ou prendre de la place sans le voir clairement.

Ce que je dois faire pour éviter ça dans l'avenir

1. **Nettoyage régulier Docker**
   - Supprimer les conteneurs, images et volumes inutilisés souvent :

     docker system prune -a –volumes

     (supprime tout ce qui n'est pas utilisé)
2. **Surveiller l'espace utilisé**
   - Avant et après une manipulation importante:

     docker system df

     docker volume ls

     docker volume inspect <volume>
3. **Sauvegarder les volumes importants**

- Pour ne pas perdre de données, exporter régulièrement ou les monter sur un dossier hors Docker (bind mount).

4. **Utilise des fichiers de logs rotatifs**
   - Configurer les conteneurs pour ne pas garder des logs trop gros (options de logging dans docker-compose).
5. **Apprends à gérer WSL sur Windows**
   - wsl --shutdown régulièrement, redémarrer Docker Desktop, compresser les disques VHDX (si possible).
6. **Structure le projet**
   - Utiliser plusieurs dossiers/projets Docker séparés pour éviter que tout soit dans un seul gros volume.

Jour 18-20 : Projet final – Stack complète :

Mission : Créer une stack de monitoring complète

- **Architecture à déployer :**

```
┌──────────────────┐  ┌──────────────────┐
│   Uptime Kuma    │  │      Zabbix      │
│  (Disponibilité) │  │   (Performance)  │
└────────┬─────────┘  └────────┬─────────┘
         │                     │
         └──────────┬──────────┘
                    │
              ┌─────▼─────┐
              │  Grafana  │
              │(Dashboards)│
              └─────┬─────┘
                    │
              ┌─────▼─────┐
              │  Dozzle   │
              │   (Logs)  │
              └──────────┘
```

- **docker-compose.yml unifié (à créer) :**
  - **Regrouper tous les services :**

    Création du docker-compose.yml regroupant tout les services (Zabbix et sa Database/Server/Frontend/Agent, Grafana, Uptime Kuma et Dozzle)

```
  GNU nano 8.3                                                    docker-compose.yml
version: '3.8'

networks:
  monitoring-net:
    driver: bridge

volumes:
  zabbix-db-data:
  grafana-data:
  kuma-data:
  dozzle-data:

services:

  # ZABBIX DATABASE
  zabbix-db:
    image: mariadb:10.5
    container_name: zabbix-db
    environment:
      MYSQL_ROOT_PASSWORD: zabbix
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix
    volumes:
      - zabbix-db-data:/var/lib/mysql
    networks:
      - monitoring-net

  # ZABBIX SERVER
  zabbix-server:
    image: zabbix/zabbix-server-mysql:alpine-6.0-latest
    container_name: zabbix-server
    depends_on:
      - zabbix-db
    environment:
      DB_SERVER_HOST: zabbix-db
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix
    ports:
      - "10051:10051"
    networks:
      - monitoring-net
```

```
  GNU nano 8.3                                                    docker-compose.yml
  # ZABBIX FRONTEND
  zabbix-frontend:
    image: zabbix/zabbix-web-nginx-mysql:alpine-6.0-latest
    container_name: zabbix-frontend
    depends_on:
      - zabbix-server
    environment:
      DB_SERVER_HOST: zabbix-db
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix
      ZBX_SERVER_HOST: zabbix-server
      PHP_TZ: Europe/Paris
    ports:
      - "8080:8080"
    networks:
      - monitoring-net

  # ZABBIX AGENT
  zabbix-agent:
    image: zabbix/zabbix-agent:alpine-6.0-latest
    container_name: zabbix-agent
    depends_on:
      - zabbix-server
    environment:
      ZBX_SERVER_HOST: zabbix-server
    networks:
      - monitoring-net

  # GRAFANA
  grafana:
    image: grafana/grafana
    container_name: grafana
    ports:
      - "3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana
    networks:
      - monitoring-net

  # UPTIME KUMA
  uptime-kuma:
    image: louislam/uptime-kuma
    container_name: uptime-kuma
```

```
    ports:
      - "3001:3001"
    volumes:
      - kuma-data:/app/data
    restart: always
    networks:
      - monitoring-net

  # DOZZLE
  dozzle:
    image: amir20/dozzle:latest
    container_name: dozzle
    ports:
      - "8081:8080"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    networks:
      - monitoring-net
```

- **Utiliser un réseau commun :**

  Docker permet de connecter plusieurs conteneurs entre eux via des réseaux virtuels. Un réseau commun est donc un réseau Docker personnalisé que tous les services partagent, ce qui leur permet de se “voir” par leur nom de conteneur (hostname).

  Par exemple :

```
networks:
  monitoring-net:
    driver: bridge
```

networks : est la section où est déclaré les réseaux Docker que l’on veut utiliser

monitoring-net : nom donné à ce réseau

driver : bridge : il représente le type de réseau (création de réseau interne isolé dans Docker qui permet aux conteneurs connectés de se voir et d’échanger des données via des adresses IP privées).

Donc, tous les conteneurs connectés au réseau monitoring-net pourront communiquer entre eux par leur nom de service (exemple : zabbix, grafana, uptime-kuma, dozzle), sans besoin de passer par des ports externes.

Objectifs : Les conteneurs peuvent se parler directement par leurs noms de service (ex : http://zabbix:8080 depuis Grafana).

Pas besoin de passer par localhost ou par des ports exposés.

Meilleure isolation et gestion des communications internes.

```
networks:
  - monitoring-net
```

```
theot@Asus04 MINGW64 ~/desktop/unif
$ docker network ls
NETWORK ID     NAME                 DRIVER    SCOPE
842cf56ec085   bridge               bridge    local
feb389ab217d   dozzle_default       bridge    local
038b2d715cd2   grafana_default      bridge    local
1308c8640492   host                 host      local
1d8602229579   none                 null      local
b08e7f67566b   unif_monitoring-net  bridge    local
e7e82e8d308a   uptime-kuma_default  bridge    local
0c4ebdabd52e   zabbix_default       bridge    local

theot@Asus04 MINGW64 ~/desktop/unif
$ docker network inspect unif_monitoring-net
[
    {
        "Name": "unif_monitoring-net",
        "Id": "b08e7f67566b7c32d60c995c7a5b522840a8eba351a8f42a7f21bd95a2daf3cf",
        "Created": "2025-06-10T10:33:29.89737754Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.22.0.0/16",
                    "Gateway": "172.22.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "0bb161c1a1330288582187d37a1c44cc97ae606eddf26462cf243b7f0014ef48": {
                "Name": "grafana",
                "EndpointID": "214f788844979f2db0adf5eb2c65c3f9274f27b0df5d8c9337526553e91524b0",
                "MacAddress": "a6:2d:4b:f9:94:6c",
                "IPv4Address": "172.22.0.4/16",
                "IPv6Address": ""
            },
```

docker network ls : permet de recenser tout les réseaux
docker network inspect unif_monitoring-net : voit tous les services connectés à ce réseau
docker exec -it grafana bash : se connecte au conteneur (ici Grafana)
ping dozzle : ping de Dozzle depuis Grafana pour assurer le réseau commun

```
theot@Asus04 MINGW64 ~/desktop/unif
$ docker exec -it grafana bash
0bb161c1a133:/usr/share/grafana$ ping dozzle
PING dozzle (172.22.0.3): 56 data bytes
64 bytes from 172.22.0.3: seq=0 ttl=42 time=1.839 ms
64 bytes from 172.22.0.3: seq=1 ttl=42 time=0.140 ms
64 bytes from 172.22.0.3: seq=2 ttl=42 time=0.189 ms
64 bytes from 172.22.0.3: seq=3 ttl=42 time=0.131 ms
64 bytes from 172.22.0.3: seq=4 ttl=42 time=0.141 ms
64 bytes from 172.22.0.3: seq=5 ttl=42 time=0.146 ms
64 bytes from 172.22.0.3: seq=6 ttl=42 time=0.103 ms
64 bytes from 172.22.0.3: seq=7 ttl=42 time=0.114 ms
64 bytes from 172.22.0.3: seq=8 ttl=42 time=0.129 ms
```

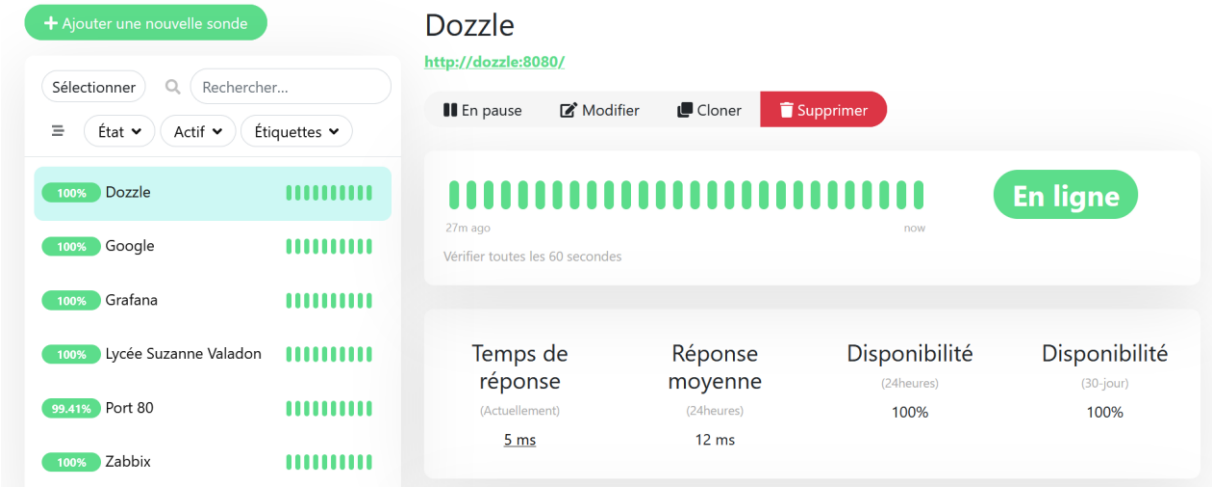
- **Gérer les dépendances :**
  En Docker Compose, les dépendances permettent de définir l'ordre de démarrage (et parfois d'arrêt) des conteneurs.
  Concrètement, si un service A dépend d'un service B, cela signifie que Docker Compose va attendre que le service B soit démarré avant de lancer le service A. C'est l'intérêt de « **deponds_on** ». Donc, cela évites des erreurs de connexion ou des services qui plantent parce qu'ils démarrent trop tôt.

```
theot@Asus04 MINGW64 ~/desktop/unif
$ docker-compose up -d
time="2025-06-10T12:33:29+02:00" level=warning msg="C:\\Users\\theot\\desktop\\unif\\docker-compos
e.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potentia
l confusion"
[+] Running 8/8
 ✔ Network unif_monitoring-net  Created                                                    0.1s
 ✔ Container zabbix-db          Started                                                    1.4s
 ✔ Container grafana            Started                                                    1.4s
 ✔ Container uptime-kuma        Started                                                    1.5s
 ✔ Container dozzle             Started                                                    1.4s
 ✔ Container zabbix-server      Started                                                    1.7s
 ✔ Container zabbix-frontend    Started                                                    2.0s
 ✔ Container zabbix-agent       Started                                                    2.0s
```

		Name	Container ID	Image	Port(s)	CPU (%)	Last sta	Actions
☐	●	zabbix-fronte	398989d68a4b	zabbix/zab	8080:8080	0.02%	5 minute	
☐	●	zabbix-server	93e0ad1885be	zabbix/zab	10051:10051	0.54%	5 minute	
☐	●	zabbix-db	8a59f898c09b	mariadb:10		0.41%	5 minute	
☐	●	dozzle	71a85c46b9ee	amir20/doz		0%	5 minute	
☐	●	uptime-kuma	21129f64fe3f	louislam/u	3001:3001	0.92%	5 minute	
☐	●	grafana	0bb161c1a133	grafana/gra	3000:3000	0.31%	5 minute	

Showing 19 items

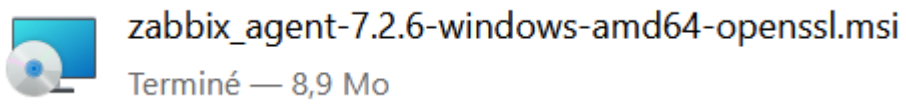
- **Configuration:**
  - **Uptime Kuma : 3 monitors :**



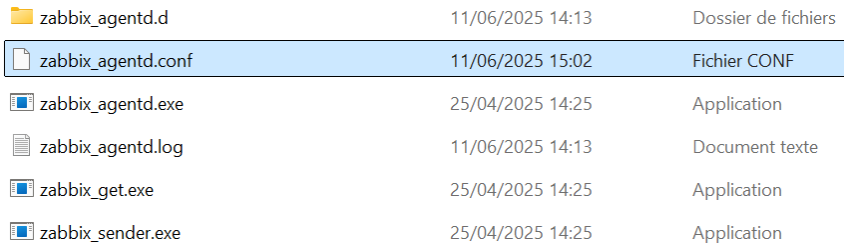
Création des 3 monitors (Dozzle, Zabbix et Grafana) avec leurs adresses de réseaux commun (ici : http://dozzle:8080/)

- **Zabbix : Surveiller le host Docker :**

Pour pouvoir surveiller le host Docker, il faut installer l'agent Zabbix :



Puis, modifier le fichier zabbix_agentd.conf et y renseigner l'adresse de la machine ainsi que le nom de l'hôte

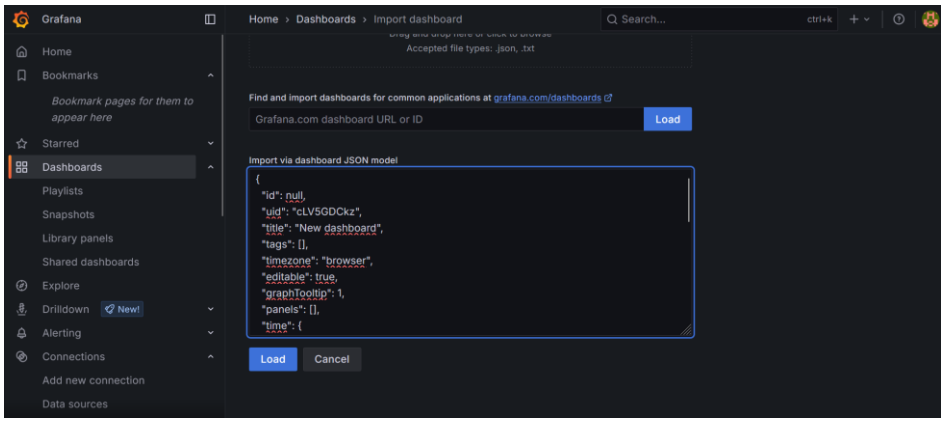
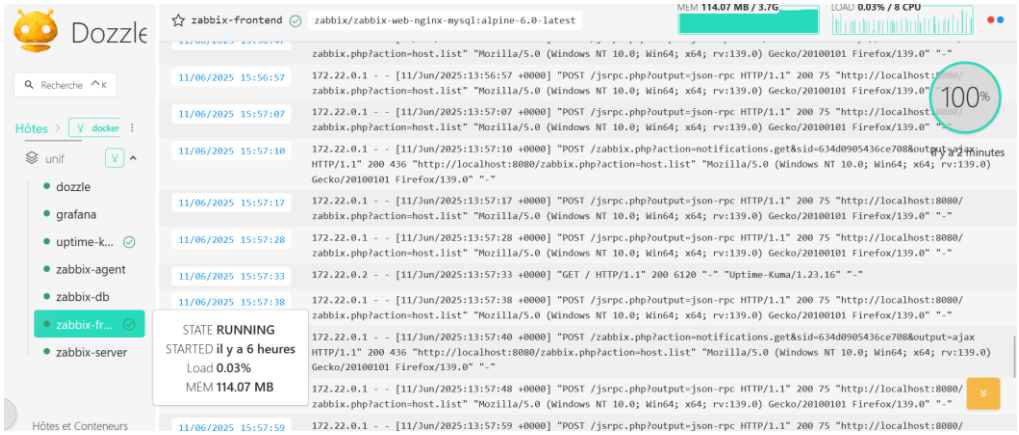


zabbix_agentd.d	11/06/2025 14:13	Dossier de fichiers
zabbix_agentd.conf	11/06/2025 15:02	Fichier CONF
zabbix_agentd.exe	25/04/2025 14:25	Application
zabbix_agentd.log	11/06/2025 14:13	Document texte
zabbix_get.exe	25/04/2025 14:25	Application
zabbix_sender.exe	25/04/2025 14:25	Application

```
ServerActive=192.168.50.17:10051
Hostname= unif
```

Puis la même étape mais sur Zabbix :

- **Grafana : 1 dashboard importé :**



- **Dozzle : Accès aux logs :**